



Member-only story

Should You Actually Try Karpathy's LLM Wiki?

The moment you feel that you can finally call up your own knowledge, that dust-covered old warehouse turns into a studio that fills you with creative energy.



Shuyi Wang

Follow

27 min read · Apr 16, 2026



264



2



Over on Knowledge Planet, one of my readers — “Jiawang Kailai” (Deng Yue) — lobbed another sharp question my way.



记往开来|邓岳

2026-04-10 16:08 阅读人数 146

收进专栏

...

王老师您好，最近AI领域的Andrej Karpathy提出了LLM Wiki的知识编译范式，核心是用LLM将个人零散的原始资料，增量编译成结构化、带交叉引用的持久化Wiki，替代传统RAG“每次查询从零检索”的模式，也重新定义了上下文工程的核心逻辑，在圈内引发了非常广泛的讨论。

您作为AI与知识管理领域深耕多年，一直为我们AI爱好者输出前沿理念与可落地实践方法的资深专家，我特别想请教您两个核心问题：

第一，从知识组织与AI落地的专业视角，您怎么看待Karpathy提出的“知识编译优于检索”这一范式？它和我们传统的个人知识管理、信息组织体系有什么本质的异同，又真正解决了过往个人知识管理中哪些长期悬而未决的核心痛点？

第二，当下各类AI Agent、记忆管理工具层出不穷，但大多各自为政，有着独立的记忆模块和知识库，很容易形成数据割裂、知识无法跨工具复用的问题。对于我们没有硬核开发能力的AI爱好者，想要基于这套理念，搭建一套不被单一工具绑定、可跨工具复用的统一个人知识体系，同时避开幻觉污染、知识失真、维护失控这些常见坑，您认为最核心的执行步骤是什么？有没有一套我们能直接上手的最小实践路径？

收起



Original screenshot is in Chinese. Deng Yue’s full two-part question is transcribed in English in the blockquote below.

If the screenshot is hard to read, here’s a transcript of the two questions:

First, from a professional standpoint on knowledge organization and AI implementation, what’s your take on Karpathy’s proposed paradigm of “knowledge compilation over retrieval”? What’s the fundamental difference between this and traditional personal knowledge management and

information organization systems, and which long-standing pain points in personal knowledge management does it actually solve?

Second, AI agents and memory management tools are popping up everywhere these days, but most of them go their own way — each with its own memory module and knowledge base, which easily leads to fragmented data and knowledge that can't be reused across tools. For an AI enthusiast like me without hardcore development skills, who wants to build a unified personal knowledge system based on this philosophy — one that isn't locked to a single tool, can be reused across tools, and avoids common pitfalls like hallucination contamination, knowledge distortion, and maintenance overhead spiraling out of control — what do you think are the most essential execution steps? Is there a minimum-viable game plan we can pick up and run with?

As it happens, Deng Yue's questions landed just as I was poking around the wiki Hermes has been spinning up for me over the past few days. This wiki was grown using the exact three-tier architecture Karpathy describes. I don't have to babysit it — creating pages, adding links, updating indexes — all the grunt work gets handed off to the `llm-wiki` workflow inside Hermes.

Karpathy published that [`llm-wiki.md`](#) GitHub gist on April 4, 2026. The `llm-wiki` workflow inside Hermes is essentially a ready-made implementation of that gist. According to my local stats, this wiki was initialized on April 8; by April 11 (the day Deng Yue asked his question), it had grown to 84 pages, fed by two folders: one with 234 articles I've published over the past few years, and another with 1,789 personal notes I've jotted down in GetNote. `index.md`, `log.md`, `SCHEMA.md` — all there.

The Big Idea

Let's tackle the first question. Boiled down, it's:

From a knowledge organization perspective, how do you view the paradigm shift from “traditional retrieval (RAG)” to “knowledge compilation (Wiki-ification)” as the better approach, and what value does it bring in solving pain points in personal knowledge management?

It’s a great question, because it touches on something I’ve been banging my head against for over a year now.

Where They Really Differ

On the surface, the most common naive RAG pipeline and Karpathy’s knowledge compilation are both ways of “getting knowledge to talk to an LLM.”

Naive RAG (Retrieval-Augmented Generation) follows a classic playbook: chunk your documents, convert them to vectors, store them in a vector database. Then, every time you ask a question, it retrieves a few relevant chunks and stuffs them into the prompt. To keep the comparison clean, I won’t get into graph-based RAG variants like GraphRAG here. [If you’re interested in GraphRAG, check out my other article on the topic.](#)

The LLM Wiki flips this around — it shifts the bulk of the work to “write time,” so that whenever you ingest a new piece of material, the LLM goes off and updates a dozen or so wiki pages, weaving the new information together with the old, building cross-links, and updating indexes.

But that’s still just the surface. For a knowledge worker, the real difference is this:

RAG starts every query from scratch, rediscovering things; LLM Wiki builds a living artifact that keeps growing.

To put it another way, RAG is like craving a stir-fry and having to pull raw chicken, vegetables, and seasoning out of the fridge and cook from scratch every single time; LLM Wiki is more like having marinated meat, simmered

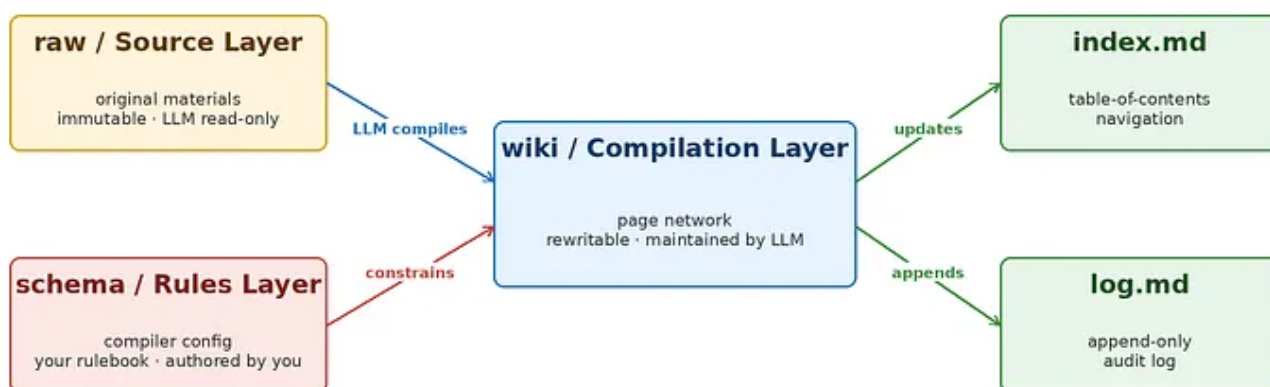
stock, and your aromatics already prepped — when it's time to cook, you're just assembling. Sure, RAG does some preprocessing too (chunking and embedding), but that kind of preprocessing doesn't build up any understanding — every query still reassembles fragments and burns compute from scratch. With LLM Wiki, most of the “thinking it through” work has been done in advance, and the artifact itself is something you can actually maintain long-term and version-control.



There's a key claim Karpathy makes in the gist that boils down to this: the grunt work of maintaining a knowledge base isn't in the reading or thinking — it's in the **bookkeeping** (categorizing, numbering, updating cross-references, building indexes). The reason wikis have generally failed in the past is that the cost of this mechanical bookkeeping scales out of control as the wiki grows. And now, the LLM happens to be able to take on the vast majority of that bookkeeping for you, updating a dozen related pages in one shot. Once that mechanical cost gets crushed, the wiki as a form becomes viable again.



The skeleton of this thing is dead simple. Karpathy breaks it into three layers.



The raw layer (original materials, read-only for the LLM) is your immutable source layer. The wiki layer is the network of pages the LLM compiles from the schema's rules — this can be rewritten over and over. The schema layer is the “rulebook” you write yourself, telling the compiler what format to follow and what to avoid. On top of that, `index.md` is the wiki's table-of-contents navigation, and `log.md` is an append-only audit log. Both are updated automatically by the LLM with every operation — you don't have to maintain them by hand.

A Seventy-Year-Old Cousin

Your first question also asks something more granular: how does this relate to traditional personal knowledge management methods? I think it's worth tying this in with Niklas Luhmann's Zettelkasten (slip-box) note-taking method, because that's what determines whether you should think of LLM Wiki as "yet another shiny new thing" or as "an automated version of a method that's been practiced for over seventy years." After all, Luhmann started doing this back in the early 1950s.

In spirit, LLM Wiki and Luhmann's Zettelkasten are close cousins; in practice, LLM Wiki takes over all the time-consuming, non-thinking parts of Luhmann's method.

Over his lifetime, Luhmann used two Zettelkästen to write out around 90,000 cards, and by the time he died he had published more than 500 books and papers. If you read his 1981 paper *Kommunikation mit Zettelkästen* (**Communicating with Slip-Box Systems**), you'll find that his starting point for designing the system lines up strikingly well with Karpathy's in the 2026 gist.

First, both reject synthesizing from scratch at query time. The reason Luhmann accumulated 90,000 cards is that he didn't trust himself, when it came time to write a book, to dig up every supporting argument from memory. Karpathy spells this out in the gist too: "the LLM is rediscovering knowledge from scratch on every question. There's no accumulation." Decades apart, both are addressing the same complaint: **on-the-fly synthesis is unreliable; you have to accumulate first.**

Second, both are "compilation-first" rather than "retrieval-first." When Luhmann wrote a permanent note, he had already thought an idea through — the permanent card is itself a compiled product. Karpathy's wiki pages are also compiled products: at ingestion time, the LLM has already cross-integrated the new source with existing pages. Both push the most

cognitively demanding work forward to “write time” rather than leaving it for “query time.”

Third, both reject hierarchy. In his paper, Luhmann explicitly rejected categorizing cards by topical hierarchy — his point, roughly, is that as soon as you systematically classify by content, you’re locking in an order decades in advance, and the growth of knowledge can’t be locked in like that. So his Zettelkasten wasn’t divided into chapters by topic; instead, he used numbers and links to weave the cards into a network. Karpathy’s wiki is the same — concept pages, entity pages, and comparison pages are connected by bidirectional links, with no folder hierarchy. When the German scholar Johannes F. K. Schmidt later studied Luhmann’s papers, he found that Luhmann’s real navigation mechanism wasn’t the numbering itself — it was a set of index pages that served as entry points. That genuinely echoes Karpathy’s idea of letting `index.md` carry the entry-point navigation.

Fourth, both treat the system as a “conversation partner.” In his 1981 paper, Luhmann explicitly called his slip box a “communication partner.” His point was that once a card collection reaches a certain volume, it starts surfacing connections you didn’t see coming — **the system starts teaching you back.** Karpathy doesn’t use that word, but he keeps emphasizing that the wiki is a persistent artifact that compounds the more you feed it. The two of them are saying the same thing.

Now that we’ve covered the parallels, let’s talk about which part of Luhmann’s method LLM Wiki replaces. Luhmann himself said that the vast majority of his time went into wrestling with the Zettelkasten, not into writing books. The time sink wasn’t the reading or the drafting — it was dealing with the slip box. Specifically: hand-writing numbers, building cross-references, updating hub cards, maintaining keyword indexes, and deciding where a new card belonged.

Every time he added a new card, Luhmann might have to go back to several older cards and write in new links by hand. The labor of 90,000 cards... sigh,

the man was a true grinder.

In my article *In the Age of AI, Should You Throw Out Your Knowledge Management Tools?*, I touched on this **80% of the grunt work** — to make your knowledge machine-readable, and findable by your future self, you have to do classification, numbering, indexing, and cross-referencing. None of this work directly **contributes to thinking**, but if you don't do it, you can't get back to what you saved. Luhmann was willing to spend most of his life doing this because he had no other choice — back in the 1960s, you couldn't reliably hand the job of automatically linking cards over to a machine.

Of course, saying that bookkeeping and thinking can be cleanly separated is a simplification. When Luhmann built links by hand, the very decision of why these two ideas are related is itself a form of thinking. But there is at least a big chunk of the work — number maintenance, index updates, cross-reference completion — that's purely mechanical. The paradigm shift of LLM Wiki is precisely about handing most of that mechanical labor over to the LLM, while also taking on some of the “thinking at the connection level.” The latter means: you have to put that thinking back during your review step. Every time the `llm-wiki` workflow inside Hermes ingests a new piece of material, it updates the cross-references on a dozen related pages in one go, appends entries to `index.md`, writes audit records into `log.md`, and assigns stable frontmatter to new concepts. Link-maintenance work that used to take Luhmann hours holed up in his study can now be compressed into a very short window.

But the 20% of Luhmann's method that actually generates thinking — LLM Wiki can't do that for you, and shouldn't. This also lines up with the integrated reading-thinking-writing approach Sönke Ahrens emphasizes in *How to Take Smart Notes*. Before you promote a fleeting thought into a permanent note, you have to make a value judgment — is this thought worth keeping? Is it sharp enough? How does it connect to the knowledge network I already have? That judgment is bound up with your research direction, your taste, your sense of which questions matter — it can't be outsourced.

In *Understanding Smart Notes on a Single Page*, I also pointed out that the number of permanent cards Luhmann would settle on each day was actually quite small. The point was never bulk hoarding — it was high-quality filtering. LLM Wiki can't make that judgment for you. What it can do is **save you the bookkeeping work that used to eat most of your time, and free you up to spend that energy on the judgments that actually matter.**

Reading this far, I know what you're thinking.

Come on — Luhmann wrote his permanent notes with his own thinking. Your “permanent notes” are just stuff the LLM synthesized on its own. How is that the same?

Great point. But who says that with this kind of wiki you have to “passively accept” — that whatever the LLM generates becomes your final permanent note?

I actually had a conversation with Hermes about exactly this question. Here's what it told me.

看一下你的 LLM Wiki Skill。

HW

我想问一下，你生成的这样一个 Wiki，我有没有后续对它进行修改的可能呢？

9:04 AM ✓

 skill_view: "llm-wiki" 9:04 AM

Henry Wang

看一下你的 LLM Wiki Skill。我想问一下，你生成的这样一个 Wiki...

可以，而且是默认支持后续修改的。

这个 llm-wiki Skill 生成的不是封闭系统，而是一个普通目录里的 Markdown Wiki。你后续有两种改法：

1. 你自己改

直接用 Obsidian、VS Code、任意文本编辑器打开就能改。

Skill 里明确写了：

"The wiki is just a directory of markdown files — open it in Obsidian, VS Code, or any editor."

2. 让我继续改

后面你可以直接说：

- 给这个页面补一段
 - 重写某个概念页
 - 合并两个条目
 - 更新 index
 - 跑一次 lint 修复坏链
- 我都可以接着维护。

但有个边界要注意：

1. raw/ 下面是原始材料层

这个层按设计是“只读/不可改”的，Skill 里写得很清楚：

"Never modify files in raw/ — sources are immutable."

也就是说：

- raw/articles/
- raw/papers/
- raw/transcripts/

H

Screenshot is Hermes replying in Chinese inside Telegram. The core of its answer is translated into English in the blockquote that follows.

It was long, so I'll just pull out the core idea:

The AI-generated wiki isn't a finished product — it's a starting point for human-AI co-editing. Your manual edits are treated by the AI as the factual basis of the current version. The AI doesn't blindly follow the original template; it evolves from your edits as the baseline. To bake your overall thinking into the system permanently, you can modify the SCHEMA (the rules file) to promote personal preferences from “local content” to “system-wide principles.”

So if your final permanent notes lack your own thinking, you can't pin that on the LLM.

Back to your question: what's the relationship between LLM Wiki and traditional note management (taking the Zettelkasten as the example)? My answer is — **it's not a replacement; it's the tool Luhmann himself would have welcomed if he were born in 2026.** Because what it's doing for Luhmann is exactly the grunt work he openly admitted was eating up most of his time. But it's not a “press this and you don't have to think anymore” button — the time you save was always supposed to go into the 20% that really matters: deciding what to read, deciding what to keep, deciding which side to believe, deciding whether an idea deserves to become a permanent card.

LLM Wiki isn't about “ditching the Zettelkasten method” — it's about “**letting every ordinary person — not just a Luhmann — actually run Luhmann's method.**” This used to be a method only full-time scholars could shoulder; now anyone willing to do the judgment work seriously can run it. I think most discussions undersell how big a deal this is.

What It Actually Fixes

I want to talk about exactly which pain points this LLM-built wiki approach solves for knowledge workers.

Pain point one: “build it but never use it.” In that article, *In the Age of AI, Should You Throw Out Your Knowledge Management Tools?*, I talked about the way knowledge assets turn into liabilities. You think you're stockpiling

assets, but most of the time they turn into liabilities: you have to maintain them, organize them, remember which app they're in. By the time you actually want to use them, you discover that the cost of opening that library and searching through it is higher than just searching the web all over again. I've used Evernote, Notion, Heptabase, Tana, Logseq... and a lot of those notes have since fallen into "deep sleep." The root cause isn't the tools — it's that I'm a lazy human who can't keep up with the maintenance.

Pain point two: "scattered pearls." The last time I seriously sat down to build a "complete index of my published articles" was 2018. I never did it again. The reason is simple — the more articles I wrote, the more labor a hand-maintained index required. Without maintenance, those articles eventually slip from "knowledge I can call up" into "stuff I can only find by memory." In *How to Build a Self-Running Knowledge Base with Claude Skill?*, I said "the published articles form a network, not scattered pearls" — and the reason I can finally turn it back into a network is precisely because tools like NotebookLM, GetNote, and LLM Wiki are now there as "compilers" stringing things together behind the scenes.

Let me drop in a screenshot of the top of my wiki's home page (`index.md`) — you'll get a sense of what I mean by "network":

Wiki Index

Content catalog for the published-articles wiki. Last updated: 2026-04-10 | Total pages: 84

Entities

[[claude-code]] — 在已发布文章中反复出现的 coding agent 主轴：代码库闭环、技能、扩展工作流。

[[chatgpt]] — OpenAI 的对话式 AI 产品，作者工作流的核心入口之一，覆盖问答、代码、数据分析、内容创作。

[[cursor]] — AI 代码编辑器，将 AI 深度集成到开发环境中。

[[elicit]] — AI 辅助文献综述工具，作者在科研写作中多次评测其效果。

[[fabric]] — 命令行 AI 工作流框架，从音视频直出文章的集成工具。

[[gemini]] — Google 大语言模型系列（前身 Bard），多模态与长上下文处理。

[[heptabase]] — 可视化知识管理工具，白板式卡片视图，知识组织与研究写作。

[[llama]] — Meta 开源大语言模型系列，本地 AI 部署与低成本推理。

[[midjourney]] — AI 图像生成工具，绘图与可视化场景。

[[mistral]] — 欧洲开源大语言模型，模型对比评测中的能力与定位。

[[notebooklm]] — Google 基于个人知识库的 AI 研究工具，Deep Research 与 Audio Overview。

[[obsidian]] — 本地优先的 Markdown 双向链接笔记工具，常被放在“重器轻用”和数据掌控权语境下讨论。

[[perplexity]] — AI 搜索与深度研究工具，基于大模型的搜索问答与结构化调研报告。

[[quarto]] — 跨格式文档发布工具，Reveal.js 幻灯与 PDF 生成。

[[readwise]] — 阅读高亮与笔记同步工具，知识管理工作流中的信息整合。

[[recraft]] — AI 绘图工具。

[[sora]] — OpenAI 视频生成模型，AI 视频生成的能力边界与应用场景。

[[stable-diffusion]] — 开源图像生成模型，本地部署与能力迁移讨论中的关键坐标。

[[tana]] — 结构化笔记与知识管理工具，Super Tag 与节点式数据库。

[[windsurf]] — AI 代码编辑器，与其他 AI 编码工具的对比。

[[zotero]] — 学术文献管理工具，科研工作流中的 AI 集成与功能演进。

[[codex]] — 高速执行与审查补位的重要代理，经常与 Claude Code 形成互补和制衡。

[[openclaw]] — 长期在线助手系统：消息入口、设备节点、网关与自动化。

[[openai]] — AI 工具生态核心提供者：GPT、Codex、DALL-E、Sora 的底层模型底座。

[[anthropic]] — Claude 系列开发商，代码能力与长上下文突出，是 coding agent 的模型底座。

[[deepseek]] — 中国团队开源模型，高性价比推理，国产模型代表。

[[qwen]] — 阿里通义千问，中文理解和代码能力突出的开源模型。

[[kimi]] — 月之暗面 AI 助手，超长上下文和中文友好性。

[[u-mind]] — AI 辅助思维导图，写作前期结构化和教学大纲构建。

[[nano-banana]] — 高分辨率 AI 绘图工具，教学插图和信息图素材生成。

[[roam-research]] — 双向链接笔记起点，影响 Tana、Logseq 等后续工具。

[[autogen]] — 微软多 Agent 协作框架，研究导向的多智能体对话。

[[github]] — 代码托管平台，Copilot/Codex 的集成入口，项目托管地。

Concepts

[[agent-workflows]] — 关于 AI Agent、工作流、长期协作与工具选择的一组核心判断。

[[ai-assisted-research]] — 选题、检索、读论文、综述、核查到写作投稿的 AI 辅助科研主线。

[[ai-coding-and-skills]] — 从氛围编程到 Claude Skills，再到端到端交付的 AI 编码与技能化路径。

[[ai-content-creation]] — 从单次对话到多 Agent 协作与端到端内容管线的创作演化。

[[ai-in-education]] — 站在真实课堂困境里思考 AI 如何重塑教学、作业与讲解方式。

[[author-methodology]] — 作者常用的工作法：真实触发、纠偏、验证、沉淀规则，再回到“你该怎么办”。

[[context-management]] — 上下文压力、隔离、规则外置与跨入口延续，是长期协作质量的关键地基。

[[harness-engineering]] — 驾驭工程：重点不在提示词，而在约束、反馈、纠偏与持续改进。

[[human-judgment-and-private-data]] — 人类写作与判断中真正难以外包的部分：欲望、体感、价值观与私有经验。

[[prompt-craft]] — 提示词不是玄学，而是从模糊需求到稳定输出的可复用约束设计。

[[remotion-video-workflows]] — 以 Remotion / Claude Skill 为代表的生产与迭代工作流。

[[wsy-writing-style]] — 从已发布文章反推出来的稳定写法：真实触发、先肯后纠、案例落地、行动收束。

Comparisons

[[hermes-claude-code-openclaw]] — 三者并非同类货架：分别回答代码闭环、消息入口与长期协作延续的不同问题。

Reader Entry

[[reader-questions-start-here]] — 按读者当下的困惑入手：靠不靠谱、总要重教、工具太多、写得没人味、该培养什么能力。

[[can-you-trust-ai-with-real-work]] — 回答“AI 到底靠不靠谱？哪些事能放心交给它，哪些地方必须自己把关？”

[[why-ai-keeps-needing-reteaching]] — 回答“我明明纠正过了，为什么 AI 还是老像第一次见面？”

[[what-should-you-still-learn]] — 回答“AI 这么强了，我还要不要学基础？力气到底该花在哪儿？”

[[tool-chaos-and-note-sprawl]] — 回答“工具越攒越多、笔记越放越散”这一类常见焦虑，强调入口、出口与回溯路径。

[[why-style-mimicry-still-feels-empty]] — 回答“明明已经模仿了风格，为什么读起来还是像没有人站在后面？”

[[is-this-new-tool-worth-migrating-to]] — 回答“新工具看起来很强，我到底该不该搬家？”

[[can-non-programmers-build-their-own-ai-helper]] — 回答“我不是程序员，还能不能把自己的 AI 助手和流程真用起来？”

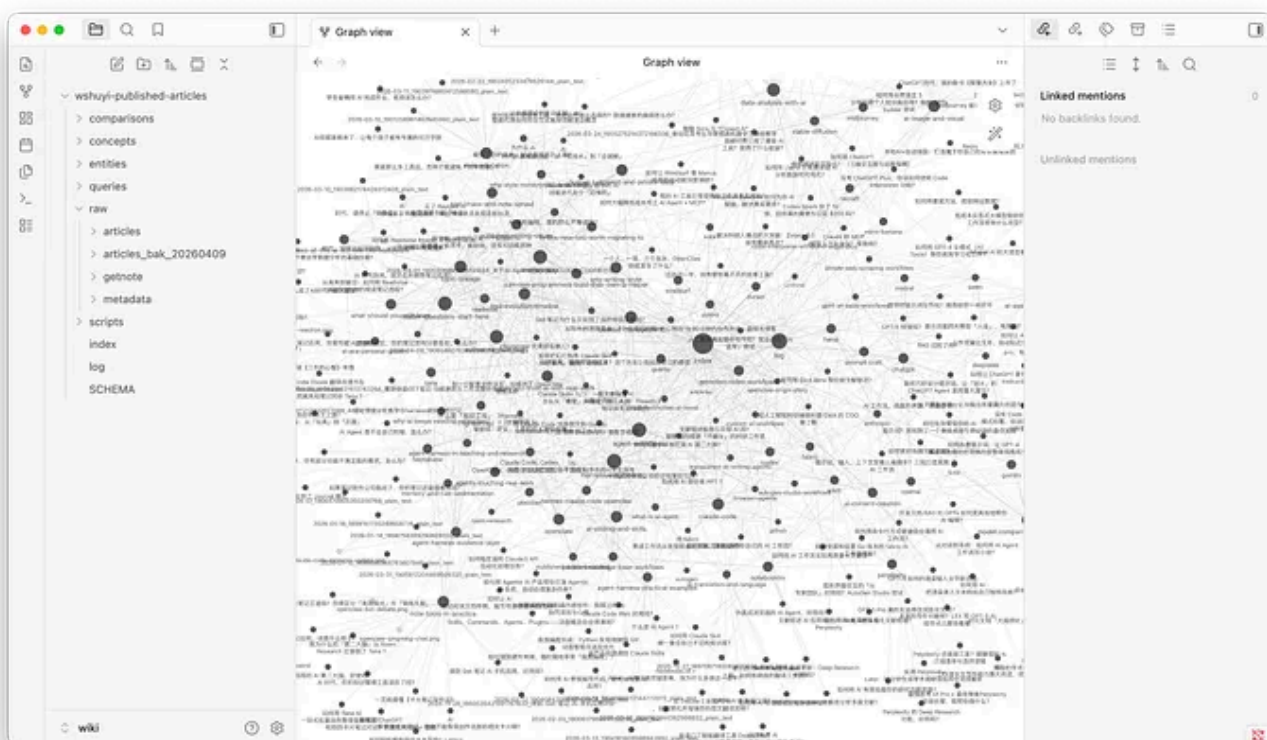
[[why-does-all-this-ai-still-feel-like-homework]] — 回答“我学了这么多 AI，为什么还是像在交作业，而不是在做真正想留下来的东西？”

Queries

Screenshot of index.md (in Chinese) — a long entry page with sections for Entities, Concepts, Comparisons, Reader Entry, and Queries, exactly the categories described in the next paragraph.

The entries you see — Entities (like claude-code, notebooklm), Concepts (like agent-workflows, prompt-craft), Comparisons, and Queries (query backfill pages) — every one of these was compiled by the LLM according to the

schema, and the pages point to each other via bidirectional links that Obsidian can render.



Pain point three: “the second you close the laptop, you can’t come up with a thing.” Some time ago, a reader named Zhang Wenru asked me four questions, the first being, “Which AI tools do you use day-to-day in your work? What specific tasks does each one handle?” The question sounds simple, but with my laptop closed I couldn’t come up with a thing — not because I haven’t used them, but because I’ve used too many tools, written too many reviews, and they’re scattered across too many articles. To answer that question well, you have to comb through years of articles and dozens of tools. That I could answer Wenru’s question and ship it within a few hours wasn’t down to memory — it was down to a workflow behind the scenes that automatically searches my article library. In a traditional knowledge management setup where I’d have to flip through hundreds of articles by hand, never mind how long it would take, the main issue is that I’d probably just give up.

Pain point four: “context drift.” You write a new article, your view has moved a step forward, a revision has happened — but the conclusion in the old

article from three months ago is still sitting in the library, and nobody's gone in to update it. The old "current view" slowly turns into the "outdated view," and nobody notifies you. Traditional PKM has no mechanism for solving this — you can only manually re-check whenever you happen to remember. In Karpathy's lint check, "stale claims" are explicitly called out as one of the health checks. My recommendation is to run lint at least once a week, just to turn "stale" into an observable signal.

These four pain points have always been there. It's not that I wasn't trying to solve them before — I just never found the right tool. Karpathy's LLM Wiki paradigm isn't magic; it's more like taking the bottleneck that traditional wikis could never break through — the cost of mechanical maintenance — and unloading a big chunk of it onto the LLM.

A Caveat

By this point you might be wondering: so as long as I get the wiki up and running, I'm all set? No. I want to flag something that's easy to miss in the tidy story Karpathy tells:

Whether this paradigm actually works for you depends a lot on what's in your raw layer.

I see plenty of people in the community discussing LLM Wiki under the implicit assumption of "I'll just toss everything into raw/ — the web pages I've clipped, the PDFs I've read, the videos I've watched, other people's paper notes." That works technically, but you'll find that the wiki you compile is "a structured aggregation of other people's views" — clean, full, well-supported, but unfortunately, **not your voice**.

I went the other route. The raw layer is, of course, just a source layer; but in my own practice, I deliberately only put things in there that I've personally digested: per my local stats, that's 234 published articles plus 1,789 personal notes I've kept in GetNote. The first set is finished output I've written; the second is the raw material of my daily thinking — but both are my voice. No

clipped web pages from elsewhere, no third-party paper excerpts. Why? Because I want this wiki to become **a map of my own thinking**, not “a watered-down version of the world’s knowledge.” As for outside information, getting it via AI is no longer hard, so there’s no need to hoard it all in my own precious wiki space.

So is Karpathy’s method worth practicing? My answer is: yes, **but only if** you treat it as a knowledge-compilation workflow with clear boundaries — not a machine that thinks for you automatically.

Putting It Into Practice

Let’s get to the second question. Short version:

Game plan: How can a non-developer beginner build a unified personal knowledge system that’s reusable across tools, low in hallucination risk, and not locked to a single platform?

This is a really practical question. I’m guessing you might already have this picture in your head: you open Obsidian, create a new vault, and stare at the blank window — where on earth do I start filling this in? Or worse, you’ve already installed five note apps, and each one has a pile of half-dead notes in it that you can’t bring yourself to delete but can’t be bothered to maintain either.

Both of those states are very common. Let’s take them one at a time.

Is This Too Hard for You?

In 2026, “I don’t know how to code” is no longer a roadblock.

The whole pipeline for my wiki — initializing the directory structure, writing the SCHEMA, ingesting new sources, compiling pages, updating indexes,

running lint checks — runs end to end on the off-the-shelf `llm-wiki` workflow in Hermes. **I haven't written a single line of code, and I haven't manually built a single page.** Of course, as I said earlier, when I do my review pass I do make edits, but at that point it's no different from editing plain text in Obsidian.

What I do is drop the materials I want ingested into the `raw/` directory, tell it to "compile this batch per the schema," look at the result when it's done, and tell it to run again if something's off. In the whole pipeline, my responsibilities are "judgment" and "sign-off"; the actual execution is on Hermes.

This isn't actually the first time I've done this. I've used other AI workflows on knowledge bases before, and the experience is consistent: you describe the goal, test the result, and make judgments; the actual execution is left to the tool. It's just that compared to the heavy-lift approach I used before, the `llm-wiki` route is much easier to pick up.

So when you ask “can a non-developer build an LLM Wiki,” my answer is: you don't need to learn to code, but you do need to learn two things — how to state your needs clearly, and how to do testing and review thoroughly.

Once you've got those two down, the next step isn't picking a tool — it's taking inventory of whether you have any material worth compiling. **The real starting point isn't “which tool should I use,” it's “do I have a pile of real, worthwhile stuff to compile?”**

What can your raw be? Let me give you some common sources: published articles you've written before (WeChat, blog, Zhihu), notes from books you've read, weekly reports and meeting minutes you've written, questions you've answered in communities, transcripts of audio and video you've recorded, and code and docs from projects you've done. All of that can go into raw.

The key is: this stuff has to be “your voice or things you’ve genuinely digested”.

The good news is, you don’t have to get it all in at once. You can start with the easiest-to-organize category of valuable content, get that pipeline working, and then layer in other types. My own approach was to feed in the 234 published articles first and run a round; once that worked, I then connected the 1,789 personal notes from GetNote — almost eight times as many — but because the previous round had already polished the schema and page templates, the second batch ran through the `llm-wiki` workflow in Hermes without throwing me any new surprises. Narrow first, then wide, piloted in stages — that beats "trying to gulp it all down in one bite."

The Workflow

Let me walk you through the work loop of Hermes’ `llm-wiki` — that is, what it actually does for me.

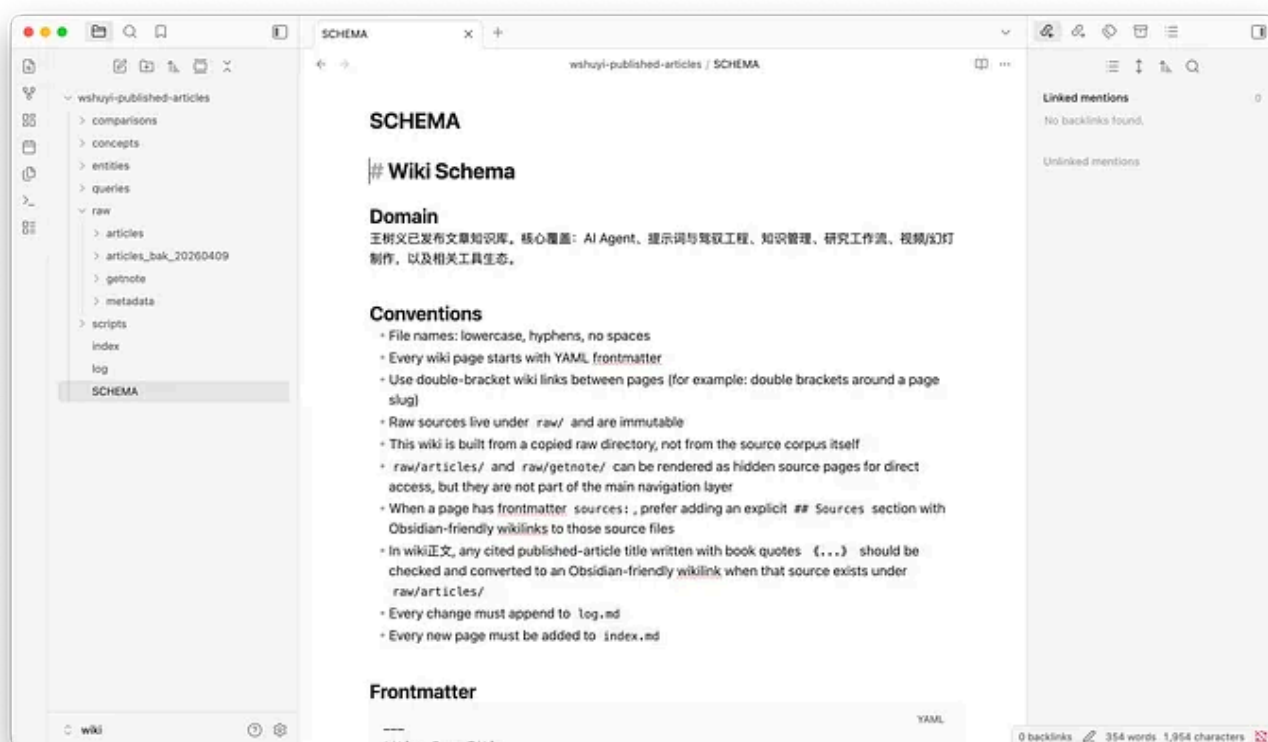
If you understand these five steps, you’ll understand what Karpathy’s design looks like in practice; even if you’re not using Hermes, swapping in a similar workflow will look roughly the same.



Step one: build the three-layer directory skeleton. The first thing Hermes’ `llm-wiki` does on initialization is set up the directory in the three-layer structure of `raw/` + the rest of the wiki layer + `schema/`. `raw/` holds the original materials, the wiki subdirectories (`entities/`, `concepts/`, `comparisons/`, `queries/`) hold the compiled output, and `SCHEMA.md` holds the rules. Those three layers are the "source layer + compilation layer + rules layer" architecture Karpathy describes — none of them can be skipped. The

whole library is just a local directory, and any editor can open it — I use Obsidian, but VS Code, Cursor, and Typora all work too.

Step two: generate SCHEMA.md. This is the easiest step to overlook, and precisely the most critical. Schema is essentially the config file the LLM uses as its compiler — it tells the LLM the directory structure of pages, the page templates, the frontmatter (the metadata fields at the top of files), the citation format, the conflict-handling rules, and which content is forbidden from being written. At initialization, Hermes' `llm-wiki` auto-generates a SCHEMA.md tailored to your domain as a starting point, and you tweak from there to match your preferences.



Screenshot of my SCHEMA.md open in an editor (Chinese annotations + English keywords). It defines Domain, Conventions, and Frontmatter — 57 lines total.

Mine is only 57 lines, but it's the single biggest lever on the whole wiki's quality — without a schema, the LLM “freelances” every time, and the wiki quickly turns into a mess of inconsistent style and chaotic structure; with a schema, it compiles everything “by the same yardstick.”

Step three: digest new sources and update multiple pages in sync. This is the first of Karpathy's three core actions. You drop a batch of new materials (articles, PDFs, notes) into `raw/`, tell Hermes to digest them, and it'll read each piece, extract the entities, concepts, and arguments inside, and update multiple related pages all at once (creating new entity pages, augmenting concept pages, appending comparison pages, updating `index.md` and `log.md`). The general benchmark Karpathy gives in the gist is that a single new source typically touches a dozen or so existing wiki pages. My wiki started with just 6 "seed pages" in the first batch — a mix of a few core concept pages, a few entity pages, and one article-catalog query page — to form the earliest layer of a core map, and only then did I expand in batches: deepening concept pages and comparison pages first, then adding theme-thread query pages and reader-question entry pages. From 0 to 84 pages in a few days. As I said before, the key isn't running fast — it's **first getting one full closed loop running**. Once you've got the loop running, that's when you find out which step is going to trip you up.

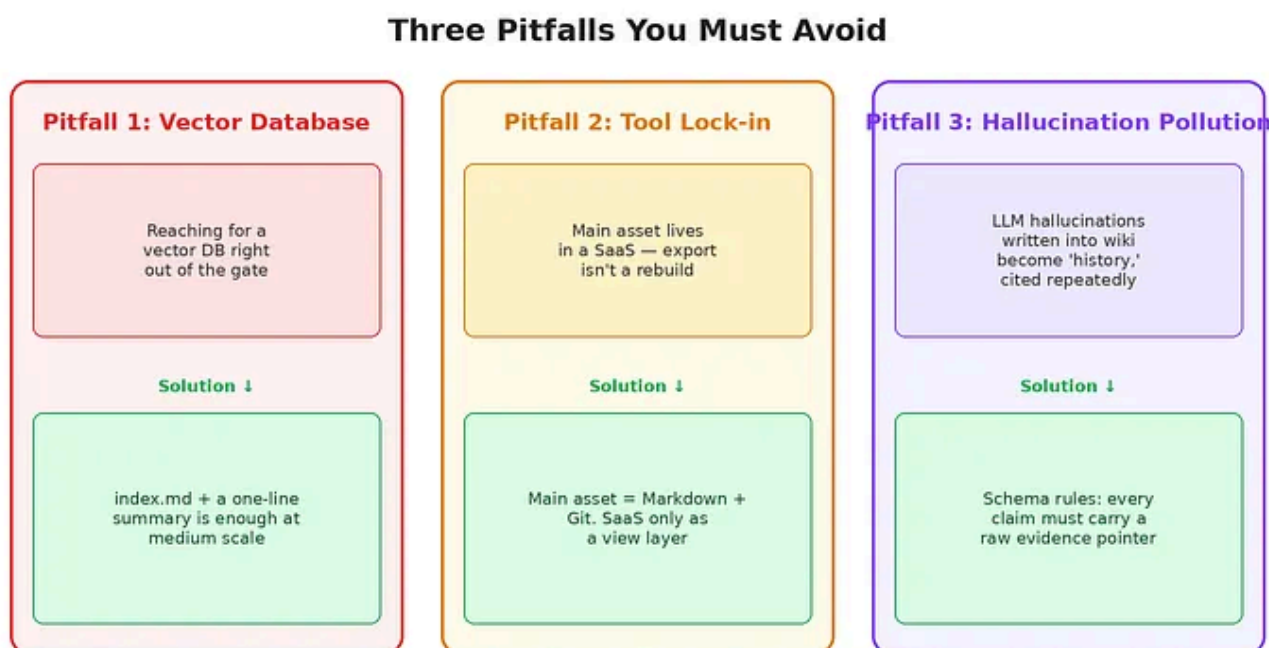
Step four: query, and backfill high-quality answers. This is Karpathy's second core action. You ask the wiki a cross-document synthesis question (for example, “lay out all the AI tools I’ve used by their roles”), and it goes searching for the answer across the network of compiled pages — note, this is searching the **already-compiled wiki layer**, not going back to `raw/` from scratch every time. The high-quality results that come out get backfilled as new pages (usually a query-type page) and slotted into the `queries/` directory. This step is critical, because it gives the wiki a **compounding effect** — every query paves the way for the next one, instead of starting from scratch every time.

Step five: lint — a regular health check. This is Karpathy's third core action, borrowed from programmer slang. Hermes' `llm-wiki` automatically scans your wiki and flags four kinds of problems: contradictions, stale claims, "orphan pages" (pages nothing links to), and missing concept pages (terms mentioned often but with no page of their own). Skip this step and the wiki will rot — old information will contaminate new information, orphan pages

will multiply, and the navigability of the whole library will gradually collapse. Whether you do lint or not is the dividing line between "a sustainable wiki" and "yet another library collecting dust."

Avoiding Pitfalls

Last, the pitfalls. Let me lay out the ones I've stepped in — plus the fixes — in one image:



Pitfall one: don't reach for a vector database right out of the gate. The most common rookie mistake is to see the words "knowledge base" and reflexively reach for a RAG pipeline — chunking, embeddings, vector database, top-k retrieval, the whole works. But my 84-page wiki to date doesn't use a single vector database. Karpathy says in the gist that for a medium-sized wiki, navigating via a one-line summary in `index.md` is already enough — you don't necessarily need that whole "fancy" RAG infrastructure. A vector database is an optimizer, not a starting point. In *Is RAG Outdated?*, I also discussed how RAG has been folded into the broader "context engineering" stack. If your material grows enough that it exceeds what a wiki can comfortably cover, RAG is still there to add later. For instance, the [gbrain](#) project shows how to

layer vector retrieval on top of a wiki, which suits cases where the volume gets too big for a pure wiki to hold.

Pitfall two: if you want the bottom line, keep your main asset as Markdown + Git. You can use Notion as a view layer, Tana as a task layer, or Dessix as a focus layer, but your core asset should be plain Markdown files with version control. Because plenty of tools will go bankrupt, change protocols, or even lock down their export. In *If Your Note App Company Goes Bankrupt, Can You Still Use Your Notes?*, I wrote about this concern, and in hindsight that worry wasn't overblown at all. Even Notion's own help page admits that exporting your content is not the same as instantly rebuilding your entire workspace; some of Roam Research's features also require extra conversion when migrated to Markdown. Not to mention certain notoriously criticized tools whose names you probably know even better. I've already written about all of that, so I won't bother bringing it up again.

What's the upside of the Markdown + Git combination? Any editor can open it, any AI can read and write it, you can drop it on GitHub at any time and migrate to a new machine, and any mistake can be rolled back. My wiki today is entirely `.md` files plus git commits — every LLM edit comes with a diff you can inspect. That's a rock-solid foundation for "not getting locked in." Even with GetNote, which I trust, I still make a point of syncing it to my local machine regularly, so the data stays fully under my control.

Pitfall three: hallucinations have to be blocked at the schema layer. The LLM doesn't just hallucinate in its answers — it'll write hallucinations **into the wiki**, and once they're in there, they become "history," and downstream queries will cite them as fact. The contamination spreads. Governing this can't wait until the wiki is built — you have to lay down rules at the schema layer from day one. Hermes' `llm-wiki` already gives you very solid default constraints on this front: the raw layer is immutable, the AI can only read but never modify it; the frontmatter of every new page must contain a `sources:` field pointing at the specific file path in the raw layer; every page creation or modification must update `index.md` and `log.md` to leave an audit

trail. These few rules baked into the SCHEMA and default behavior put three locks on the LLM — without raw evidence, you can't fill in the `sources:` field; without filling in `sources:`, you can't pass the page template's acceptance check. If you don't lay down these three, sooner or later the LLM will toss off a sentence that sounds reasonable but actually has no source, and the next time you query, you'll step on a mine.

Speaking of hallucinations, you might be thinking of an even bigger question: even if the pipeline runs smoothly, how do I make sure the facts in the wiki are actually trustworthy? That brings us back to the “traffic-light principle” from my article *Should You Throw Out Your Knowledge Management Tools?*. That article was about the boundaries of knowledge management in the AI era, and it still holds in the LLM Wiki context. The green-light zone can be fully outsourced to the LLM: summary generation, index updates, link completion, formatting, orphan-page checks — these are the bulk of the compilation pipeline, and you should just let them run quietly. The yellow-light zone calls for you and the LLM to spar: contradiction adjudication, concept merging, deprecation decisions — these are the things you sit down with during weekly lint and walk through personally. And the red-light zone is absolutely off-limits to outsourcing: writing core facts, value judgments, final sign-off — the moment you outsource this layer, your wiki loses the core feature of “your voice” and turns into a secondhand restatement of generic knowledge.

In other words, no matter how new the LLM Wiki paradigm is, the iron rule of the “cognitive handshake” still holds. You’re upgrading from scribe to reviewer, but you can’t turn into someone who washes their hands of it entirely. The more the AI handles for you in the green-light zone, the more important your judgment becomes in the red-light zone. If you don’t internalize this mindset, even a wiki that’s up and running will gradually drift off course.

Getting Started

If you want to try this today, my advice is: don't be greedy. Pick one category of raw you already have on hand — say, 20 published articles, or a stack of meeting minutes you've written — then stand up the three entry points: `raw/`, `SCHEMA.md`, and `index.md`. Run a minimal ingest pass once. Then ask only the two most valuable questions, like "which themes have I been writing about over and over the past few years?" and "which old articles can actually be strung back together?" Finally, do one round of lint and check for stale claims, orphan pages, and obvious contradictions. Getting this minimum closed loop running first matters far more than trying to cover the whole library on day one.

Who is this approach a better fit for? People who already have a stable batch of raw material on hand, who are willing to keep their main asset as Markdown, and who are willing to come back periodically and look at the schema and the lint. Who isn't it a great fit for, at least for now? People who don't have anything resembling raw yet and just want to one-click dump the entire internet into it, or who have no intention of doing follow-up review. Because what you'll build that way isn't your own knowledge system — it's more of a drift-prone heap of material. To dig deeper, the original material most worth starting with is Karpathy's [`llm-wiki.md`](#) [`.gist`](#).

Wrapping Up

Back to the question Deng Yue opened with — is Karpathy's LLM Wiki actually worth following?

My answer is: yes, but don't think of it as "yet another new tool."

Its real significance isn't "inventing a new technology" — honestly, you can find echoes of this idea in the Zettelkasten method and in the even earlier Memex vision. The point is that it solves **a class of pain points traditional knowledge management could never crack: the cost of mechanical maintenance**. When Vannevar Bush wrote *As We May Think* in 1945 and

sketched out the Memex, the value of the idea was beyond doubt. It's just that, looking back from today, he had already laid out the direction of a personal knowledge base and associative trails, but he hadn't yet called out maintenance cost as the central bottleneck the way we do now.

Today, the LLM can shoulder these heavy loads. Once the cost of mechanical bookkeeping gets driven way down, you start daring to do the kind of long-term accumulation you never dared to do before: a complete index of your published articles, cross-references between concepts, multi-year tracking of how your views have evolved, systematic archives of reader questions. It's not that you didn't want to do this stuff before — it's that you couldn't manage it. At least not for a lazy person like me.

It's not that Karpathy's post is so radically novel — the more down-to-earth reality is: **the modern knowledge worker finally has a helper that can take on the grunt work for us, and this helper is willing to play by my rules.** In practical terms, that helper is the `llm-wiki` workflow inside Hermes — it turns Karpathy's design into a flow you can run directly, and all I have to do is feed it real material and sign off on the results.

You don't have to write code, but you do need a stash of information worth saving (raw), a clear set of rules (schema), a helper that can put those rules into practice, the narrowest possible pilot (start with 20 pages), a weekly checkup (lint), and one red line that can never be outsourced — the vetting of core facts and the final judgment have to come from you. Put all of those together, and you've got a personal knowledge compilation system that isn't locked to any single tool, compounds over the long term, and doesn't rot easily.

The moment you feel that you can finally call up your own knowledge, that dust-covered old warehouse turns into a studio that fills you with creative energy.

I hope you'll get to feel the joy of AI-assisted knowledge management soon too.

If you find this article useful, please hit the [Applaud](#) button.

If you think this article might be helpful to your friends, please share it with them.

Feel free to [follow my column](#) to receive timely updates.

Welcome to [subscribe to my Patreon column](#) to access exclusive articles for paid users.

To watch video content, please subscribe to [my Youtube channel](#).

My Twitter: [@wshuyi](#)

Further Reading

- [• NotebookLM + Alma: Mining Answers from a Knowledge Base — Why I Take This Extra Step](#)
- [• In the Age of AI, Should You Throw Out Your Knowledge Management Tools?](#)
- [• The Knowledge Worker's Portable Power Tool](#)
- [• Local AI + Auto-Capture: Building Your Own Knowledge Management System](#)
- [• Claude Skill Snapshots: Adding an “Undo Button” to Your AI Skill Iteration](#)



Written by Shuyi Wang

2.2K followers · 319 following

Follow



PhD in Information Science. Associate Professor at Tianjin Normal University.
Former Adjunct Faculty at UNT. First Prize Winner of HackNTX 2018.

Responses (2)



Christian Broberg

What are your thoughts?

See all responses

